Fluidodinamica computazionale dei sistemi propulsivi

Introduzione al linguaggio Fortran

A. Ferrero, M. Errante

Politecnico di Torino, DIMEAS

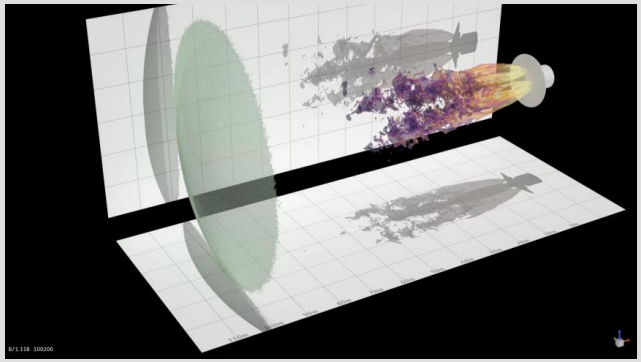
Perchè il Fortran?

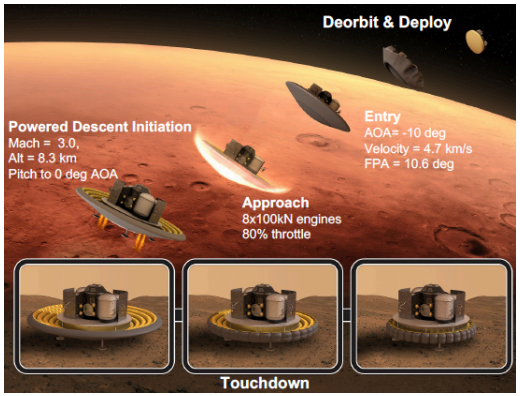
Langley Research Center

2018-2021 Summit Campaigns

Retropropulsion for Human-Scale Mars Landers

- FUN3D static simulations of 10 species/19 reactions (15 PDEs) an meshes of ~7 billion elements
- O_2/CH_4, rocket engines in Martian CO_2 atmosphere
- Broad range of spatial and temporal scales
- Two-day runs on 16,000 V100s: equivalent of several million CPU cores (billions of CPU core-hours total)
- - 90 GB of asynchronous I/O every 30 seconds for two days yields ~1 PB/run
  - 60 TB migrated from ORNL to NASA Ames daily





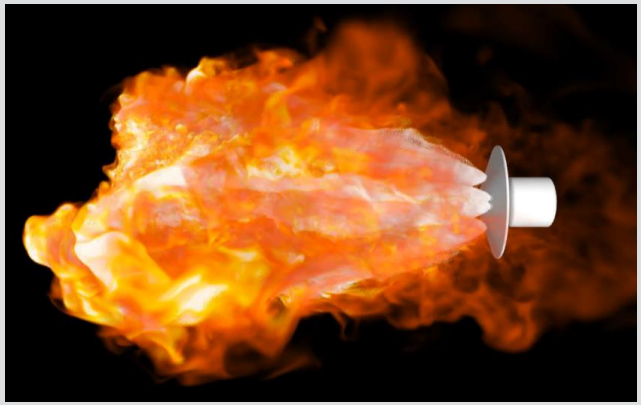
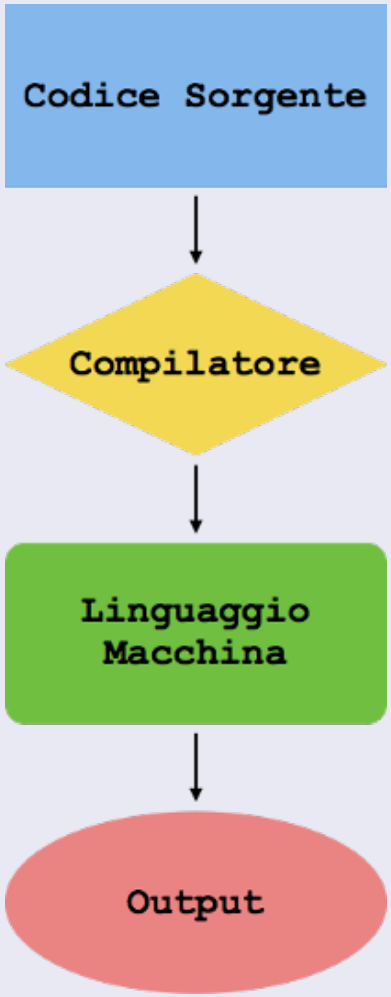


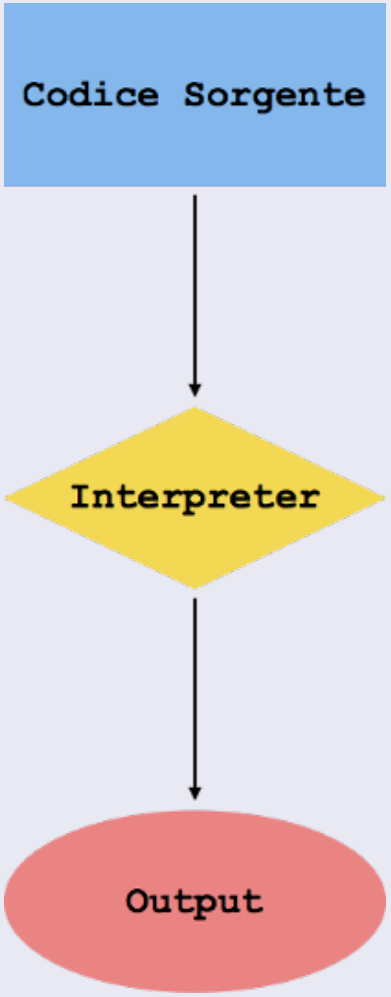
Figura: Slide workshop su FUN3D. FUN3D è una suite di strumenti di CFD, sviluppata in Fortran dalla NASA per l'aeronautica per la tecnologia ed esplorazione spaziale.
https://www.bigcompute.org/big-compute-22/the-impact-of-emerging-architectures-on-engineering-and-leadership-class-computational-fluid-dynamics

Linguaggi compilati e interpretati (1/2)

Linguaggio compilato



Linguaggio interpretato



Linguaggi compilati e interpretati (2/2)

Linguaggio compilato

Pro:

- Ready to run
- Basso running time
- Codice sorgente privato

Contro:

- Poco flessibile
- Dipendente dall'architettura
- Extra step: compilazione

Esempi:

- Fortran, C, C++

Linguaggio interpretato

Pro:

- Closs-platform
- Semplice da testare
- Semplicità di debug

Contro:

- Traduzione di ogni comando
- Running time maggiore
- Codice sorgente pubblico

Esempi:

- Python, Matlab, JavaScript

Fortran

FORmula TRANslator: linguaggio pensato per calcoli numerici

- Facile da apprendere
- Sintassi vettoriale simile a Matlab (operatore :)
- Controllo intrinseco della dimensione degli oggetti (non disponibile in C)

Evoluzione del linguaggio (principali versioni):

- FORTRAN 77: Largamente usato in passato e ancora presente in molte librerie
- Fortran 90: tipi di dati derivati, allocazione dinamica
- Fortran 2003: programmazione a oggetti analoga al C++
- Fortran 2008: parallelismo basato su coarray
- Fortran 2018: miglioramenti nel parallelismo e nell'interoperabilitá con C/C++

Regole di sintassi

- Non ci sono differenze tra maiuscole e minuscole
- Commenti con !, a capo con &
- Operatori condizionali:
  - $<$: *.lt.* (less than)
  - $\leq$: *.le.* (less or equal than)
  - $>$: *.gt.* (greater than)
  - $\geq$: *.ge.* (greater or equal than)
  - $==$: *.eq.* (equal to)
  - $!=$: *.ne.* (not equal to)
- Tipi di dati:
  - real (o real((4)) : variabile reale in singola precisione
  - double precision(o real(8)) : variabile reale in doppia precisione
  - integer: variabile intera

Input e Output

- Un programma comunica con l'esterno tramite le unità di input/output, che generalmente sono la tastiera, il video, i file su disco.
- Generalmente un programma legge il valore delle variabili dall'esterno (INPUT), le elabora, quindi scrive i risultati verso l'esterno (OUTPUT).

OPEN e CLOSE

L'istruzione **open**:

```
open (unit = 100, file = "input.dat")
open (100, file = "input.dat")
```

- `unit` indica il numero di unità associato al file: sarà lo stesso numero da indicare come `unit` nelle istruzioni di `read` o `write`.
- `file` indica il nome del file a cui si deve accedere.

L'istruzione **close**:

```
close (unit = 100)
close (100)
```

- chiude l'unità e libera il numero associato ad essa.

READ e WRITE

L'istruzione **read**:

```
read (unit = 100, fmt = *) valore_input
read (100, *) valore_input
```

- `unit` rappresenta l'unità su cui si vuole operare per leggere dati.
- `fmt` determina il formato dei dati da leggere o scrivere.
  `fmt` = * indica formato libero (scelta consigliata).

L'istruzione **write**:

```
write (unit = 100, fmt = *) valore_output
write (100, *) valore_output
```

Per entrambe le istruzioni, `unit = *` indica la periferica di default:

```
read (*,*) input !legge il valore di input da tastiera
write (*,*) output !scrive il valore di output a video
```

Cicli DO

Equivalente al ciclo FOR in Matlab e C

```
do i = 1,N
    ...
end do
```

Costruttori condizionali

Costrutto completo:

```
if (condizione) then
    SERIE DI ISTRUZIONI A
else
    SERIE DI ISTRUZIONI B
end if
```

Costrutto semplificato per una sola operazione:

```
if (condizione) ISTRUZIONE SINGOLA
```

Vettori e matrici

In Fortran il conteggio degli indici parte da 1 per default (esiste peró la libertá di usare il range che si preferisce, addirittura con indici negativi).

Dichiarazione di un vettore di real con 10 elementi (indici standard tra 1 e 10):

```
real,dimension(10):: vettore
```

Dichiarazione di un vettore di real con 10 elementi (indici tra 0 e 9) :

```
real,dimension(0:9):: vettore
```

Dichiarazione di una matrice di real con 20×10 elementi:

```
real,dimension(20,10):: matrice
```

Vettori e matrici allocabili

Il Fortran permette di dichiarare array senza specificarne a priori le dimensioni, che verranno poi assegnate in corso di esecuzione.

Esempi di dichiarazione:

```
real,dimension(:,:),allocatable :: matrice
real,dimension(:),allocatable :: vettore
```

In seguito alla dichiarazione di un array allocabile per riservare memoria si usa l'istruzione:

```
allocate(matrice(dim1,dim2))
allocate(vettore(dim))
```

Somma tra matrici

Quattro modi equivalenti di calcolare una matrice c come somma di due matrici a e b (notare similitudine con Matlab):

```
do j=1,10
    do i=1,20
        c(i,j)=a(i,j)+b(i,j)
    end do
end do
```

```
c(1:20,1:10)=a(1:20,1:10)+b(1:20,1:10)
```

```
c(:,:)=a(:,:)+b(:,:)
```

```
c=a+b
```

Somma dei primi N numeri

```
program test
    implicit none
    integer :: i,N
    real :: somma

    write(*,*) "N ?"
    read(*,*) N

    somma = 0
    do i =  1,N
        somma = somma + i
    end do

    write(*,*) "somma = ", somma

end program test
```

Somma dei primi N numeri pari

```
program test
    implicit none
    integer :: i,N
    real :: somma

    write(*,*) "N ?"
    read(*,*) N

    somma = 0
    do i =  1,N
        if((mod(i,2)).eq.0) somma = somma+i
    end do

    write(*,*) "somma = ", somma

end program test
```

Somma numeri da file di input

```fortran
program test
    implicit none
    integer :: i,val_in,somma

    somma = 0
    open (1, file = "input.txt")
        read (1,*)
        do i =  1,5
            read (1,*) val_in
            somma = somma + val_in
        end do
    close(1)

    write(*,*) "somma = ", somma

end program test
```

File `input.txt`:

```
Dati di input
1
1
2
3
5
```

Subroutine

- In Fortran si possono creare SUBROUTINE (e anche funzioni) che svolgono compiti specifici
- Facilitano lo sviluppo e il debug del codice
- **ATTENZIONE**: tutti i parameteri sono **SEMPRE** passati per riferimento, quindi se vengono modificati all'interno della subroutine risulteranno modificati anche nel codice chiamante all'uscita dalla suboutine

Esempio SUBROUTINE(1/2)

Sorgente del MAIN:

```
program test
    implicit none
    integer :: i,N
    real :: somma

    N = 7

    call compute_sum(somma,N)

    write(*,*) "somma = ", somma

end program test
```

Esempio SUBROUTINE(2/2)

Sorgente della SUBROUTINE:

```
subroutine compute_sum(somma,N)
    implicit none
    integer :: N,i
    real :: somma

    somma = 0
    do i =  1,N
        somma = somma+i
    end do

end subroutine compute_sum
```

Moduli

I moduli permettono di condividere variabili (o definizioni di subroutine) tra varie subroutine:

```
module variabili
    implicit none
    save
    real :: R = 287.d0
end module variabili

subroutine compute_pressure(P,rho,T)
    use variabili
    implicit none
    real :: P,rho,T

    P = rho*R*T

end subroutine compute_pressure
```

Tipi derivati

I tipi derivati permettono di creare *dati strutturati*, raggruppando in un unico oggetto dati eterogenei:

```
module strutture
    implicit none
    type elemento
        integer :: n_nodi, tipo_elm
        real :: area
    end type elemento
end module strutture

program main
    use strutture
    implicit none
    type(elemento),dimension(:), allocatable :: ele
    integer:: i,n_ele

    allocate(ele(n_ele))
    do i =  1,n_ele
        call compute_area(i)
    end do

end program main
```

RACCOMANDAZIONI

Usare **SEMPRE** il comando IMPLICIT NONE all'inizio del MAIN e in ogni SUBROUTINE o FUNZIONE.

Fare attenzione alle divisioni tra interi: se a=2 e b=4 con a e b di tipo INTEGER allora a/b=0.
Per ottenere 0.5 bisogna fare un cast o moltiplicare per 1.: (1.*a)/b